2026

Low-Level HW Digital Systems II

LAB MANUAL & COURSEWORK

VASILIS F. PAVLIDIS – PROFESSOR, ECE DEPT.
EVANGELOS TZOUVARAS - PHD STUDENT

AUTH | Electronics Laboratory

Contents

1 Introduction 2
2 Simulation Tools 2
2.1 Questa – Intel FPGA Edition 2
3 System Verilog Example 4
3.1 Setting up the Simulation Environment 4
3.1.1 Design under test 4
3.1.2 Simulation tool 7
4 Coursework 8
4.1 Floating Point Representation 8
4.2 Addition with Floating Point Numbers 11
4.3 Floating Point Adder – Design 12
4.3.1 Main module 13
4.3.2 Sign calculation (inside the main module) 14
4.3.3 Exponent calculation module 14
4.3.4 Mantissa calculation module 14
4.3.5 Normalization module 15
4.3.6 Rounding module 16
4.3.7 Post-round normalization (inside the main module) 17
4.3.8 Exception module 17
4.4 Floating Point Adder – Verification 19
4.4.1 Testbench 19
4.4.2 System Verilog Assertions 21
5 Deliverables for Submission 21
6 References 22
7 Appendix 23

1 Introduction

This document describes briefly the prerequisites required to set up the tools for compiling, simulating, and verifying the circuits you will design as part of the coursework for this course. The document is divided into three sections. The following section includes some guidelines regarding the simulators you can use and some concise instructions on how to install and use a simulator offered for free for Intel's FPGAs. Section 3 describes the simulator setup, along with a System Verilog example. The final section describes the coursework you need to complete, and the deliverables.

2 Simulation Tools

A well-established tool for RTL simulation has been the Modelsim simulator [1] of Mentor-Graphics (now part of Siemens). Unfortunately, the student edition of this great tool is not available any longer. Thus, you need to seek another simulator that effectively has integrated Modelsim in their environment. There are a few available options, which you can use for free for student projects. These options include, for example, the Quartus tool from Intel [2] and Libero® SoC Design Suite from Microchip [3].

We recommend the use of Quartus provided on [2], which supports both a Windows and a Linux version. Unfortunately, there are no simulation tools that we know of that presently support Mac OS. Thus, Mac users need to find a workaround. In the following, we succinctly describe how to install and use the Intel simulator. Note that we cannot offer you any technical support on this topic (I managed to get the tool up and running, hence, I am confident you will have no problems either).

2.1 Questa – Intel FPGA Edition

Follow the steps below to install the Quartus simulator for RTL simulation. Alternatively, you can use any other tools you like (even the one you used in the last semester but make sure that all work fine).

- Go to [2], click on Individual files in the Download Section, and select to download the Questa*-Intel® FPGA Edition. Please check you have sufficient disk space to proceed with the installation. Although the tool requires about 1 GB during the installation process you may need much more (possibly up to 30 GB [4]). Also, check the memory requirements of the simulator (as with any other tool).
- During the installation process, you SHOULD select Intel FPGA **Starter** Edition as only this edition offers a free license for 1 year.
- Once the tool installation is complete, a process that takes quite long (tens of minutes depending on your computer), you need to acquire a license file. Please visit the Intel® FPGA Self Service Licensing Center (SSLC) and sign up using your **institutional email** (please do not use your personal email, e.g., Gmail).
- After registration, you will receive a confirmation email from Intel. Using this email, log on SSLC and click on the "Sign up for Evaluation or Free Licenses" tab and select Questa*-Intel® FPGA Starter Edition, as depicted in Fig. 1, where you edit the number of seats to one. Click on "Get license".

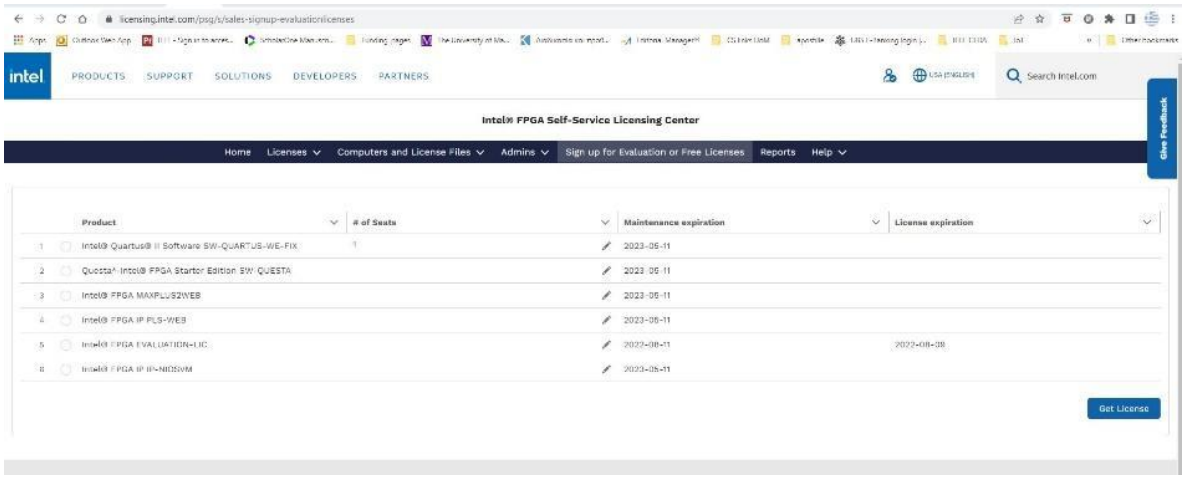

Fig. 1. SSCL Screen

- You will need to provide the NIC of your computer (physical or MAC address of your Ethernet or wireless card, which consists of 12 hex digits) that is used to identify your computer (execute `ipconfig -all` on the Windows command prompt to obtain this hex number). Please note that you can support up to 3 devices with the same license file. In this case, you need to install the software on all three machines. In addition, note that you cannot run the software remotely but only locally with this type of free license.
- Typically, it takes some time (nominally up to 12 hrs) to register your account on SSCL, although you have received the activation email. So, allow some time for this to happen before you attempt to check out a license.
- By selecting the proper type of license, you will receive an email where the license file with the extension .dat will be attached. Save this file to a desired location, typically, some folder within the top-level directory `intelFPGA`. This file is used by the license daemon of the tools, located in the path C:\intelFPGA\21.1\questa_fse\win64.
- An environment variable should also be added to the system. For Windows, right-click on Start, select System, Advanced System Settings, Environment Variables, and create a New variable called LM_LICENSE_FILE and provide the path where you stored the license file you received from Intel (*.dat). Click OK. An example of this setup is shown in Fig. 2.
- You can now start using the software. Also, use the Intel® FPGA Software Installation and Licensing manual available on the elearning course webpage. Note that you are interested in the Intel FPGA **Starter** Edition.

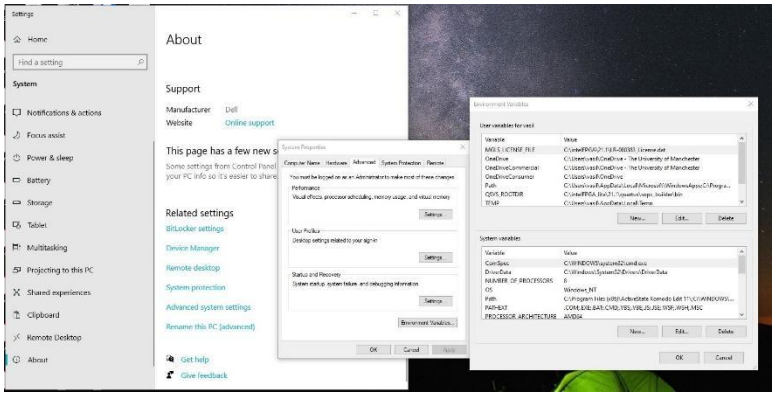

Fig. 2. Setting up the environment library for Questa – Intel FPGA simulator.

3 System Verilog Example

This section contains an example of a circuit described in System Verilog (SV) and tested with typical testbenches and SV Assertions (SVA), in Questa simulator. In subsection 3.1 the simulation environment and an example is described, while in subsection 3.2 the tool commands are provided.

3.1 Setting up the Simulation Environment

The basics of setting up a circuit for functional simulation are first described. Specific steps that you need to follow in the Questa-Intel simulator to examine SVA are also provided. To simulate a design, you need a testbench (similar to the methodology you used in Low-Level HW Digital Systems I in the 7th semester). In addition to this file, another file that contains all the SVA, you have created for the design (DUT) or circuit (CUT) under test, must be included for simulation. This is done through the `bind` command described in the course. This binding process is schematically shown in Fig. 3.

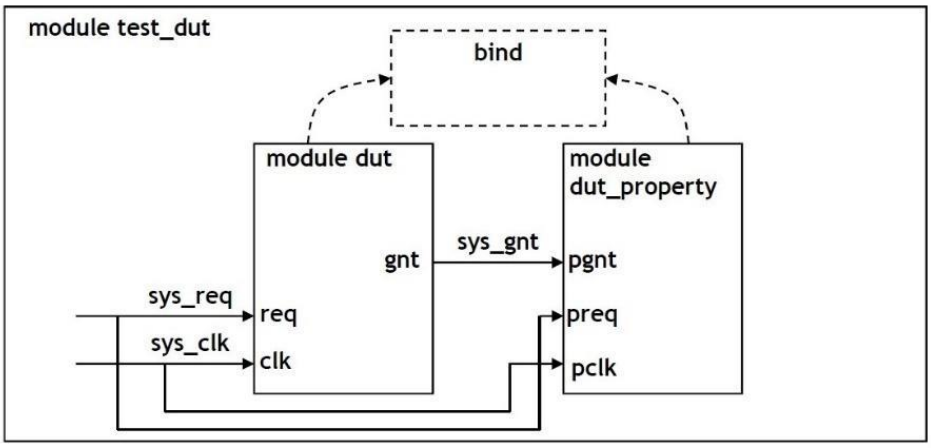

Fig. 3. In the module test_dut, the dut and dut_property modules are bound. The test_dut is the testbench that contains all required information to test the module dut. SVA are contained in the dut_property module.

In the following, the code for each of these three modules is provided such that you understand the methodology of binding modules, using SVAs, and producing an appropriate testbench. Note that you need to read and understand how these files work altogether, using also the examples and material you have been provided during lectures.

3.1.1 Design under test

A simple behavioral Verilog model that acts as the DUT driving a simple req/gnt protocol follows. The code in the text box below includes different assertions that may be desired to check the functionality of the DUT. Read this code carefully combined with the taught material in the lectures. Note the presence of macros (defined with the `ifdef elsif endif` structure), which can help you choose the property that is checked each time. You may opt to completely ignore such macros.

Also, note that any command starting with (`) is a compilation directive rather than part of the RTL code and, hence, of the DUT.

```
module dut_property(pclk,preq,pgnt);
input pclk,preq,pgnt;

`ifdef no_implication
property pr1;
  @(posedge pclk) preq ##2 pgnt;
endproperty
preqGnt: assert property (pr1) $display($stime,,,"\t\t %m PASS");
          else $display($stime,,,"\t\t %m FAIL");

`elsif implication
property pr1;
  @(posedge pclk) preq |-> ##2 pgnt;
endproperty
preqGnt: assert property (pr1) $display($stime,,,"\t\t %m PASS");
          else $display($stime,,,"\t\t %m FAIL");

`elsif implication_novac
property pr1;
  @(posedge pclk) preq |-> ##2 pgnt;
endproperty
preqGnt: assert property (pr1) else $display($stime,,,"\t\t %m FAIL");

property pr2;
  @(posedge pclk) preq ##2 pgnt;
endproperty
cpreqGnt: cover property (pr2) $display($stime,,,"\t\t %m PASS");
`endif

endmodule
```

```
module dut(clk, req, gnt);
input logic clk,req;
output logic gnt;

initial
begin
  gnt=1'b0;
end
initial
begin
@(posedge req);
  @(negedge clk); gnt=1'b0;
  @(negedge clk); gnt=1'b1;
@(posedge req);
  @(negedge clk); gnt=1'b0;
 @(negedge clk); gnt=1'b0;
end
endmodule
```

The binding process and the stimuli used in this example are presented in the following textbox. Invest as much time as required to understand what this code does, how inputs are stimulated and how real-time report messages are produced during the simulation. You can remove the $finish command, as this command pops up a window that asks to quit the tool, not just the simulation.

```
module test_dut;
bit sys_clk,sys_req;
wire sys_gnt;

/* Instantiate 'dut' */
dut dut1 (.clk(sys_clk),.req(sys_req),.gnt(sys_gnt));

bind dut dut_property dutbound (clk,req,gnt);
//
// You need to know the names of the ports in the design and the property module
// to be able to bind them. So, here they are:
// Design module (dut.v)
// ----------------------
// module dut(clk, req, gnt);
//          input logic clk,req; //          output logic gnt;
// Property module (dut_property.sv)
// ---------------------------------
//module dut_property(pclk,preq,pgnt); //input pclk,preq,pgnt;

always @(posedge sys_clk)
  $display($stime,,,"clk=%b req=%b gnt=%b",sys_clk,sys_req,sys_gnt);
always #10 sys_clk = !sys_clk;

initial
begin
      sys_req = 1'b0;
  @(posedge sys_clk) sys_req = 1'b1; //30
  @(posedge sys_clk) sys_req = 1'b0; //50
  @(posedge sys_clk) sys_req = 1'b0; //70
  @(posedge sys_clk) sys_req = 1'b1; //90
  @(posedge sys_clk) sys_req = 1'b0; //110
  @(posedge sys_clk) sys_req = 1'b0; //130

@(posedge sys_clk);
@(posedge sys_clk); $finish(2);
end
```

3.1.2 Simulation tool

You can always compile and simulate these files using the Questa Intel HDL simulation tool as you did in the coursework of Low-Level HW Digital Systems I and, therefore, no detailed description will be provided. In general, you need to

- Create a new project (give a name of your preference) and fill in the fields on the displayed window.
- Add existing files or create a new file through the tool editor. When you add existing files, think about whether you want to copy the files to the working directory (easier for command line reference to the files) or keep them in the reference directory (Fig. 6 and Fig. 7 in Appendix).
- Once you have successfully compiled all related files (Fig. 8 and Fig. 9 in Appendix), you are ready to start the simulation. Click on Simulate -> Start Simulation. On the window that pops up, expand the "work" library and pick the files you want to simulate as shown in the related figure below (Fig. 10 in Appendix).
- Click on the optimization options and select full (or custom) visibility (Fig. 11 in Appendix) that allows retrieving all signals in the design hierarchy to view these signals on the waveform window. Select the component dut and add its signals to the waveform (Fig. 12 in Appendix).
- Click on the Run icon and you can simulate your circuit for 100 ns or, whichever, other option you prefer (Fig. 13 in Appendix).
- To activate the properties since these are encapsulated within macros, you need to explicitly define these arguments (one or more) during the compilation of the dut_property.sv file. The compile command (vlog) is of the form:

```
vlog -sv dut_property.sv +define+no_implication
```

- The argument "-sv" means compile with SystemVerilog options and "+define" defines a macro, in this case, the "no_implication" property. More macros can be simultaneously defined by adding more macros to the command. The tool returns a warning in this case that multiple macros are defined but you can safely ignore this.
- Restart the simulation (not the tool!) and you will see that some assertions now appear. Information on assertions can be found by clicking on the Assertions tab right at the bottom of the waveform subwindow (Fig. 14 in Appendix).
- **Tips**: type help <command> in the command line of the simulator to get help with any command. The simulator also provides auto-complete on any command or path. Use the Up arrow in the Transcript window to see the history of executed commands as you proceed with different tasks through GUI or typed commands.
- Click Help -> Questa sim – Intel FPGA Edition – Bookcase to find a rich set of manuals. Use these with caution and at your own discretion as they contain hundreds of pages!

4 Coursework

This section describes the tasks that you need to complete as part of the laboratory exercises. The Floating Point Representation, based on IEEE-754 standard is described in subsection 4.1. The equivalent mathematical addition of two Floating Point numbers is outlined in subsection 4.2. A briefly description of the requested design is given in subsection 4.3, while the verification process of the design is presented in subsection 4.4.

Each student should individually submit a report (**strictly in pdf format**) with all the deliverables as required in Section 5.You are required to design with SV and verify with a testbench and SVA the circuit presented in the following subsections. You must:

- Produce the SV code
- Test the code correctness through a testbench
- Verify correct functionality of the circuit with SVA

4.1 Floating Point Representation

All floating-point numbers consist of three parts:

1. A *sign* - Indicates whether the number is positive (sign = 0) or negative (sign = 1)
2. An *exponent* - This is the power of 2 (binary) that the mantissa is multiplied with to get the actual value.
3. A *mantissa/fraction* - The "fractional" portion of the number. There is an implied leading **1 (hidden bit)** (we consider only normal numbers).

The sizes of the exponent and mantissa may differ based on the precision used. A few different precisions can be seen in Figure 1. For the module under design, you will be using only the **single precision (32-bit)**, which has an 8-bit long Exponent and a 23-bit long Mantissa.

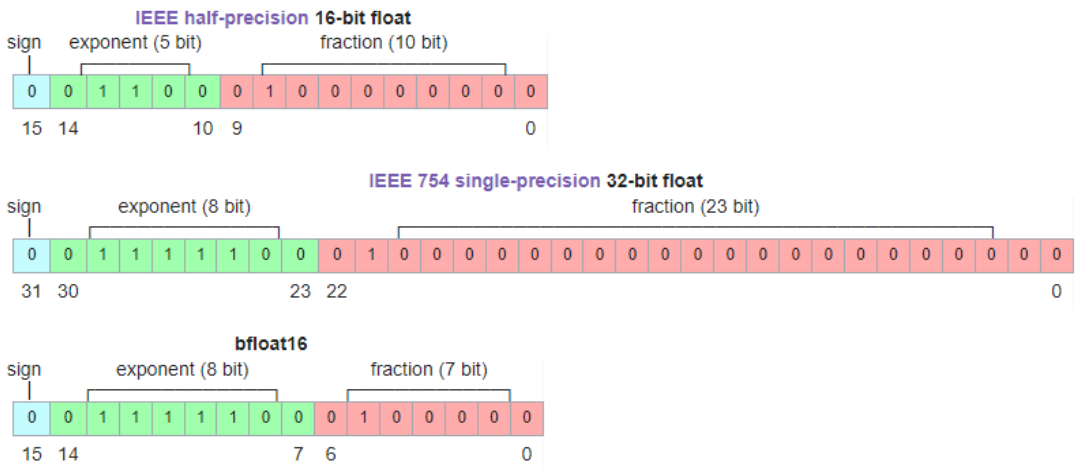

Figure 1: Three different types of floating point precisions. From top to bottom the 16-bit half, the 32-bit single, and the 16-bit bfloat16.

The actual value of a floating point number can be acquired through the representation above as can be seen in Figure 2 specifically for single precision. The exponent is always subtracted by a **bias**, in our case this number is **equal to 127**.

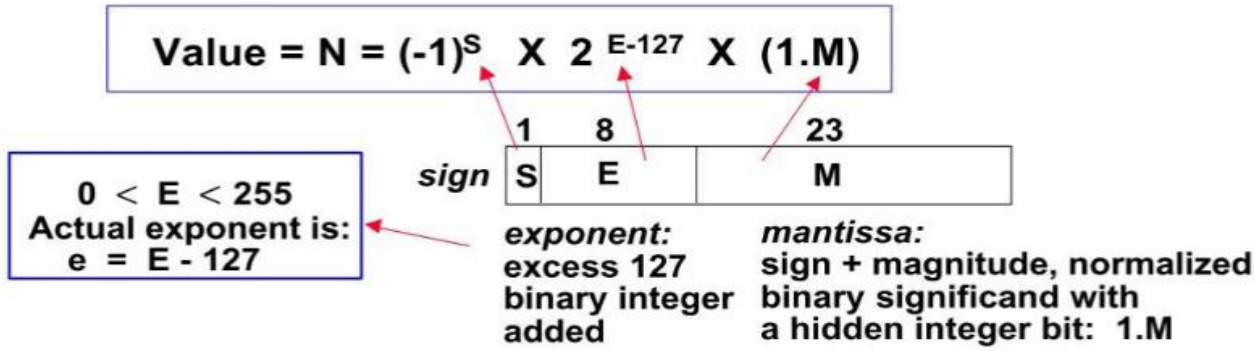

Figure 2: The actual single floating point value can be found using the sign, exponent, and mantissa bits of the representation.

Based on the values in each of the three parts, the numbers can be categorized as shown in Figure 3.

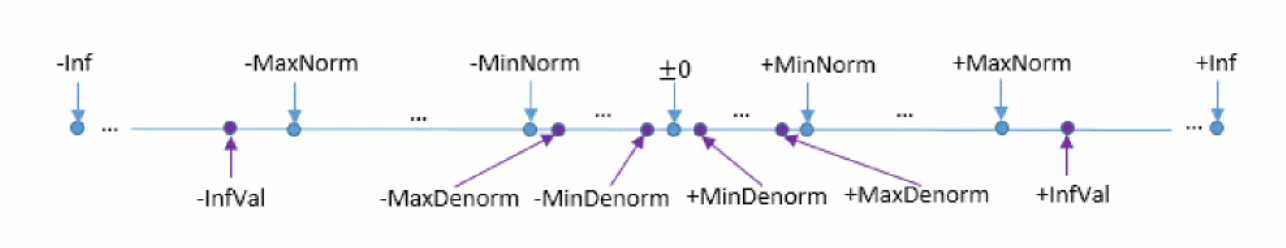

Figure 3: The location of all the special floating point numbers on a 1D line. The order is the following: -NaNs, -Inf, -Normals, -Denormals, -Zero, +Zero, +Denormals, +Normals, +Inf, +NaNs

Table 1 shows the pairing between each category and the representation in the respective floating point number regardless of the precision.

As can be seen in Table 1, a 0 Exponent and 0 Mantissa represent the value positive/negative zero depending on the sign value. Likewise, an Exponent with all bits equal to 1 and Mantissa equal to 0 represents the positive/negative infinities based on the sign.

All the numbers that have the bits of the Exponent equal to 1 but the Mantissa has at least one bit equal to 1 are considered to be NaNs. A denormal is a number that has all the bits of the Exponent equal to 0 and a Mantissa with at least one bit equal to 1. All the other numbers are considered to be normals and represent a value based on Figure 2.

Sign (s)	Exponent (exp)	Significand (sig)	Value	Short Name
s	0	0	$(-1)^s 0$	±Zero
s	111...111	0	$(-1)^s$Infinity	±Inf
s	111...111	$0 < sig$	Not a Number	NaN
s	0	$0 < sig$	$(-1)^s\, 2^{1-bias}\, 0.sig$	Denormal
s	$0 < exp < 111...111$	sig	$(-1)^s\, 2^{exp-bias}\, 1.sig$	Normal

Table 1: Special values for floating point numbers. 'S' denotes the sign, 'Sig' denotes the Mantissa, and 'exp' denotes the Exponent. Bias is equal to 127 for single precision.

In this coursework, we will implement a partial support of IEEE-754 standard (in bibliography is referred as IEEE compliance 0), where:

- **All denormals will be considered as zeroes (+/- Zero, based on sign bit)**
- **All NaNs will be considered as +/- Infinity**

Table 2 also shows some special floating point numbers, which can be seen in Figure 3 as well. The MinNorm and MaxNorm values represent the lower and upper bounds of the positive and negative normal numbers. Likewise, the MinDenorm and MaxDenorm values represent the lower and upper bounds for the denormals.

Sign	Exponent	Significand	Value	Short Name
s	1	0	$(-1)^s\, 2^{1-bias}$	±MinNorm
s	111...110	111...111	$(-1)^s\, 2^{bias}\,(2 - 2^{-sig_width})$	±MaxNorm
s	0	1	$(-1)^s\, 2^{1-bias-sig_width}$	±MinDenorm
s	0	111...111	$(-1)^s\, 2^{1-bias}\,(1 - 2^{-sig_width})$	±MaxDenorm

Table 2: The upper and lower bounds of the normal and denormal numbers. 'S' denotes the sign, 'sig_width' denotes the Mantissa width, and bias is equal to 127 for single precision.

Each floating point module (*e.g.*, an FP-adder or an FP-multiplier), except for the result, produces an 8-bit status value. Each of these bits indicates whether the result has a special-corner value or whether the output resulted from rounding or from underflow/overflow. The status bits according to the IEEE-754 standard, are presented in Table 3 (Theoretically, the zero and infinity bits are not defined in the standard, but are used widely in FP modules).

Bit	Flag	Description
0	Zero	$z = \pm Zero$
1	Infinity	$z = \pm Inf$
2	Invalid	$F = NaN$
3	Tiny	$\lvert F\rvert \neq Zero/Inf/NaN$ and $\lvert F\rvert < MinNorm$
4	Huge	$\lvert F\rvert \neq \pm Inf/NaN$ and $\lvert F\rvert > MaxNorm$
5	Inexact	$\lvert F\rvert \neq Zero/Inf/NaN$ and $F \neq z$
6	Unused	Reserved to 0
7	Div by 0	Division by zero, reserved to 0 otherwise

Table 3: Meaning of each bit of the output status flag

<u>Warning:</u> **z** is the component's output whereas **F** is the calculated result before it gets rounded to z.

4.2 Addition with Floating Point Numbers

Starting with two numbers, according to the IEEE-754 standard, the addition must produce a result in the same format, as presented in Equation 1.

$$(-1)^{S_A} * 2^{e_A} * 1.M_A + (-1)^{S_B} * 2^{e_B} * 1.M_B = (-1)^{S_Z} * 2^{e_z} * 1.M_Z \qquad (1)$$

Firstly, the sign bit of the result z must be calculated. When the two operands have the same sign, the resulted sign-bit will have the same number, but when the two operands have opposite sign, the resulted number will have the sign of the greater number.

- $If\ S_A = S_B: S_Z = S_A = S_B,$
- $Else\ if\ S_A \neq S_B: S_Z = S_A, |A| > |B|, else\ S_Z = S_B$

Next step is the alignment of two exponent parts. The goal is to find the greater exponent of two operands and to factor this out. Suppose that $e_A > e_B$ and d is the difference between them:

$$d = e_A - e_B \quad (2)$$

Thus, the Equation 1 can be written as:

$$(-1)^{S_A} * 2^{e_A} * 1.M_A + (-1)^{S_B} * 2^{e_A} * 2^{-d} * 1.M_B = (-1)^{S_Z} * 2^{e_A} * (1.M_A + 2^{-d} * 1.M_B) \quad (3)$$

Based on Equation 3, we can understand that the exponent, before the normalization of the final result, will be equal to e_A (in general equal to the larger exponent between the two operands).

The term $(1.M_A + 2^{-d} * 1.M_B)$ will be the new fractional part. To perform the addition, firstly we need to **align** the second mantissa (left shift by d). The new fractional part will be equal to:

$$M_{temp} = \pm 1.M_A \pm y.xx \ldots xx \qquad (4)$$

<u>Warning:</u> *The "addition" of two mantissas depends on the sign of two operands. Thus, the signs must be taken into consideration.*

It is obvious that the result from the addition (4) can be in the range: [0-4). The final mantissa (including the leading one bit), must have the format 1.xx….xxx. Therefore, a **normalization** is needed.

If the resulting mantissa is greater than two, (*e.g.*, 11.xx....xxx), a right shift by one position is needed:

$$11.xx \ldots .xxx = 1.yy \ldots yyy * 2^1 \quad (5)$$

If the resulting mantissa is smaller than 0, (*e.g.*, 0.zz…xxx), a left shift is needed to normalize the mantissa. The number of shifts is depending to the number of **leading zeroes**.

$$0.zz \ldots xxx = 1.yy \ldots yyy * 2^{-z} \qquad (6)$$

The remainder exponent parts due to normalization need to be integrated into the exponential part.

The addition of two mantissas (with the leading bits) after the alignment, can drive to a mantissa with more than 24-bits (23-bits fraction + 1-bit the leading one). It is very important to preserve the LSBs and use them to increase the accuracy of the operation. After the alignment and the normalization, in

the resulting mantissa we keep 3-extra bits to be used in rounding (guard, round and sticky bits). The guard bit will be the 25th-bit, the round bit will be the 26th-bit and the sticky bit will be result from **OR** operation between the remainder LSBs. Based on the input rounding variable (*e.g.*, IEEE_near, IEEE_zero, away_zero etc.), and based on the ***grs*** bits, the final mantissa will be remain the same or will be increased by one.

Except for the typical calculations, special treatment is needed when the operands have special values (*e.g.*, +Inf – Inf *etc.*). These cases, are handled in exception handling module.

4.3 Floating Point Adder – Design

The first task of the coursework is to write a floating point adder, in System Verilog. The adder will consist of six modules, the **main module**, the **exponent calculation module**, the **mantissa calculation module**, the **normalization module**, the **rounding module**, and the **exception module**. Also, an extra small sub-module, a leading zero counter, must be implemented to be instantiated inside the normalization module.

A wrapper module (fp_adder_top) is given to be used as a top-level for the other modules. The wrapper contains the input and output registers of the circuit. The relationship between the wrapper and the main module is presented in Figure 4. The entire logic inside the main module is **combinational.**

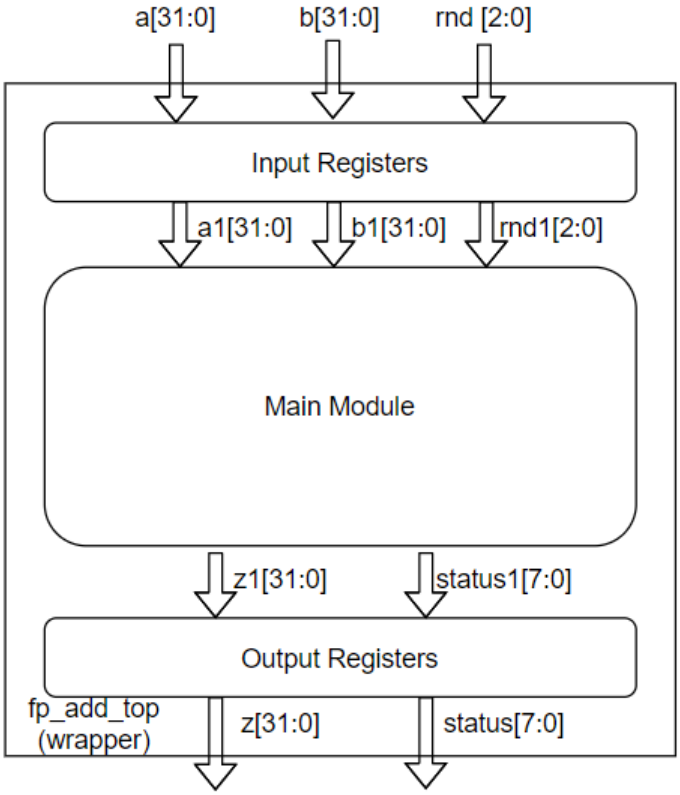

Figure 4: The relationship between the main module and the wrapper

4.3.1 Main module

The block diagram representation of the main module can be seen in Figure 5. You need to create the **fp_adder.sv** file which will be the main module. The main module of the adder will have three inputs and two outputs. The size of the two inputs (**a,b**) and the resulting output (**result**) of the adder are **32-bit** long (single precision). The size of the **round** input is 3 bits long. The **status** output is 8-bit long where each bit has a different meaning based on the operations that happen inside the adder. The meaning of each bit for the status signal can be found in Table 3 above.

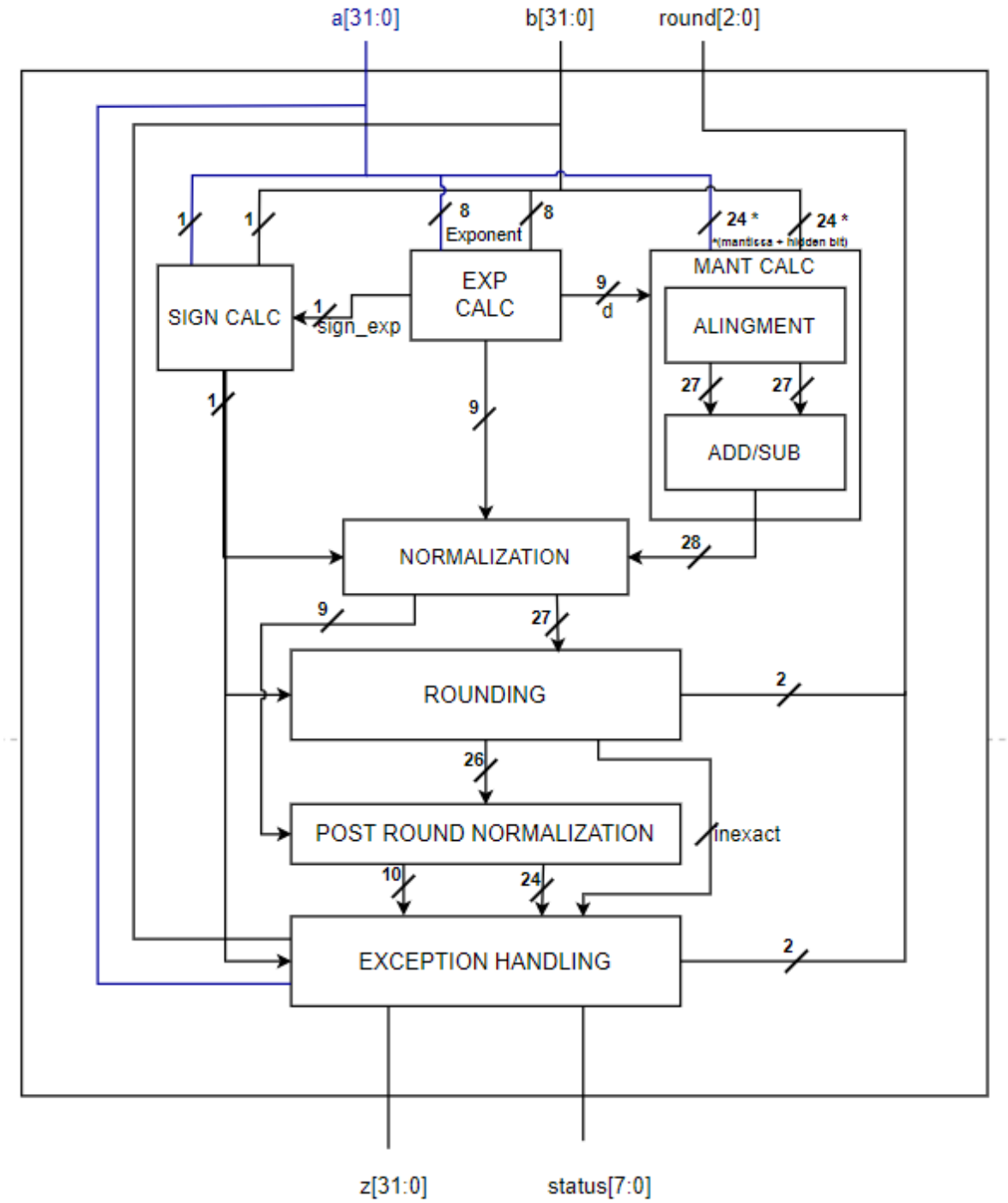

Figure 5: The block diagram of the main module of the floating point adder

The main module (Figure 5) will consist of the following 8 stages in order:

1) Floating point number **sign calculation**
2) **Exponent calculation** (instantiation of exponent calculation module)
3) **Mantissa calculation** (instantiation of mantissa calculation module)
    a) Mantissa alignment
    b) Mantissa addition
4) Truncation and **normalization** (instantiation of normalization module)
5) **Rounding** (instantiation of rounding module)
6) **Post-round normalization** and
7) **Exception** handling (instantiation of exception module)

4.3.2 Sign calculation (inside the main module)

The sign calculation takes into consideration the sign of operands ***a*** and ***b***. If the two signs are equal, then the calculated sign will be the same. In other cases, the calculated sign will be equal to the larger number.

*Note: In most of fp-adders another input signal, the operand (**op**), is used to define if the operation is addition or subtraction. Thus, the op must be taken into consideration in the sign calculation. In our design, for simplicity, we consider that the subtraction is done when the two operands have different signs only.*

4.3.3 Exponent calculation module

The exponent calculation module (**exponent_calc.sv file)** will take as input the two 8-bit exponents of operands ***a*** and ***b***. The output of this module will be the 8-bit maximum exponent (***max_exp***), the 9-bit exponent different (***exp_diff***) – the absolute value of difference between two exponents – and the sign of the exponent difference (***sign_exp_diff)***, which will be equal to 1 if the exponent a is larger than the exponent b. The block diagram of the exponent module is presented below.

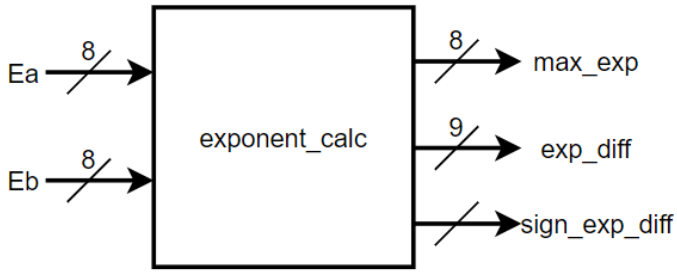

Figure 6: The block diagram of the ***exponent_calc*** module

4.3.4 Mantissa calculation module

The mantissa calculation module is responsible to perform the alignment between the mantissa part and then the addition (or subtraction). The inputs of this module are the following:

- ***exp_diff***: the 9-bit absolute difference between exponents
- ***mant_a***: the mantissa of operand ***a*** **(including the leading one bit)**
- ***mant_b***: the mantissa of operand ***b*** **(including the leading one bit)**
- ***sign_exp_diff***: the sign of the difference between exponents

- ***sa***: the sign bit of operand ***a***
- ***sb***: the sign bit of operand ***b***

The block diagram of the ***mant_calc*** module is depicted in Figure 7. The alignment process is performed in the smaller number (indicated based on ***sign_exp_diff*** signal). The aligned mantissa is calculated after the right shift by d-positions (***exp_diff***). During the alignment none of the LSBs must be lost, thus an intermediate signal of 49-bits must be declared. From the LSBs, only last three bits will be kept (the ***guard, round*** and ***sticky***). The guard bit will be the 25th bit, the round will be the 26th bit and the remainder will go to the sticky bit via a bitwise **OR**.

*Warning: If the d is larger than the 48 positions that are utilized for shift, this means that the **sticky bit** will be for sure **equal to one**.*

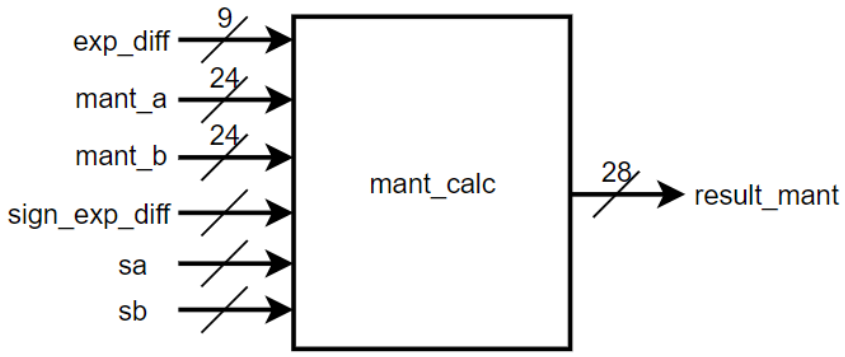

Figure 7: The block diagram of the ***mant_calc*** module

The aligned mantissa now will have a length of 27-bits (a leading-one, 23-bit mantissa part and three extra ***grs*** bits). To perform correctly the addition between the two mantissas, a concatenation is needed to the larger mantissa, by padding three zero bits at the end (this means that the ***grs*** bits of this significand part with be equal to zero).

The last part is the addition (or subtraction), between the two 27-bit significand parts. Since the sign is calculated separately from the mantissas, subtraction **ALWAYS** involves subtracting the smaller part from the larger one. The addition of two 27-bit numbers can lead to overflow, thus the output (***result_mant***) will have 28-bit length to handle the overflow case.

4.3.5 Normalization module

The normalization module (***norm_adder.***sv) is responsible to truncate and normalize the mantissa to have a final format, based on IEEE-754 standard (with MSB equal to 1, as leading bit, and 23-bits as fractional part). The module takes as input the 8-bit ***max_exp*** (calculated exponent), and the 28-bit ***result_mant*** from the ***mant_calc*** module, and produces the normalized exponent (9-bit ***norm_exp***) and the normalized mantissa (27-bit ***norm_mant***). The top level block diagram of the norm_adder module is presented in Figure

As it has been explained above, after the mantissa addition/subtraction, the result will be in the range [0, 4), which means that will have the format:

- $\mathbf{1}\boldsymbol{x}\,.\,\boldsymbol{xxx}\ldots\boldsymbol{xxx}$, if result is greater or equal than 2
- $\mathbf{1}.\boldsymbol{xxx}\ldots\boldsymbol{xxx}$ **,** if the result is in the range [1,2) – already normalized
- $\mathbf{0}.\boldsymbol{zzz}\ldots\boldsymbol{xxx}$, if the result is smaller than 1

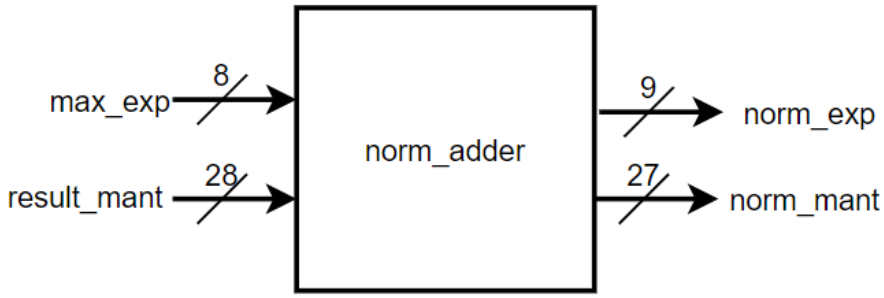

Figure 8: The block diagram of the ***norm_adder*** module

In the first case (where the MSB of the ***result_mant*** is equal to 1), a right shift is needed. Which will lead to an increment of the exponent by one. In the third case, a left shift of the mantissa is needed, until the leading-bit to be equal to one. Therefore, a leading zero counter must be implemented to measure the number of the leading zeroes, and thus the number of left shifts. Also, the ***norm_exp*** must be re-calculated accordingly.

Warning: The lost bit from right shift need to be taken into consideration in the sticky-bit (a corner case that must be taken into account)

4.3.5.1 Leading Zero Counter

The leading zero counter (***lzc.sv*** module) will take as input the 28-bit ***result_mant*** and will return a 5-bit variable with the number of ***leading_zeroes***.

4.3.6 Rounding module

The rounding module (***round_adder.sv***) take as input the 3-bit round variable, the 26-bit normalized mantissa (including the grs bits) and the calculated sign bit of the result. Then, the module produces the 25-bit round mantissa (including the leading-one and one extra bit for overflow detection), and the inexact bit, which indicates if the result has undergone rounding. **<u>Warning:</u> The rounding module will extract the 24-bit mantissa (without the *grs*) bits or the 24-bit mantissa increased by one.**

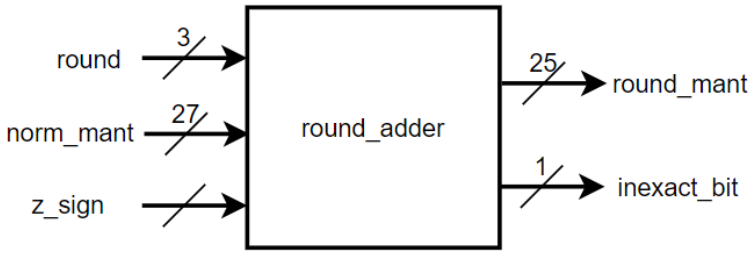

Figure 8: The block diagram of the ***round_adder*** module

The rounding is performed based on rounding value as presented in Table 4.

Note: According to IEEE-754 the supported rounding modes contain also the near_up and away_zero cases. Different architectures implement slightly different rounding modes. For example, the RISC-V architectures support the IEEE basic rounding methods and the near_maxMag and round_to_odd cases. In this coursework we will examine rounding modules that are described in the table above.

Round Type	round [3:0]	Description
IEEE_near	3'b000	Round to the nearest value. If equal near, round to the nearest even number
IEEE_zero	3'b001	Round towards zero
IEEE_ninf	3'b010	Round to -Infinity
IEEE_pinf	3'b011	Round to +Infinity
near_maxMag	3'b100	Round to the nearest value. If equal near round to the number with the maximum magnitude

Table 4: Rounding types and description

The equal near value means that the guard bit is equal to 1 and the round and sticky bits are equal to zero. For the implementation of the rounding module, a **package *round_pkg*** is requested to be constructed. The package will contain the mapping of round values into the corresponding names and will be used into the rounding and the exception handling module to increase the readability.

4.3.7 Post-round normalization (inside the main module)

After the rounding, the extracted mantissa has 25-bit length, because after the increment by one, overflow can be occurred. Thus, a post-round normalization is needed if the MSB of ***round_mant*** is equal to one, in a similar way with the normalization module.

Another significant part in the floating point calculations is the overflow and the underflow detection. The addition (or subtraction) calculations can lead to **overflow**, if the calculated exponent cannot fit in the 8-bit value, or can lead to **underflow**, if after the subtraction the exponent value is smaller or equal to zero. These to flags need to be calculated here to be fed into the exception handling module, which follows the calculations.

After the calculations, an intermediate result, the ***z_calc*** is calculated, which contains the calculated sign-bit, the 8-bits of the calculated exponent (if we consider that there is no overflow or underflow), and the calculated mantissa (23-bit, **without the hidden-bit**).

4.3.8 Exception module

The exception handling module is responsible to handle all the corner cases (*i.e.*,operation with specific value, overflow and underflow), and to assign values into the status-bits. It takes as input:

- ***a***: the 32-bit ***a*** operand
- ***b***: the 32-bit ***b*** operand
- ***round***: the 32-bit ***round*** variable
- ***z_calc***: the 32-bit ***z_calc*** intermediate result
- ***overflow, underflow*** and ***inexact_bit***: The produced flags during the operation

Based on these inputs it produces the 32-bit output ***result*** and the 6 status-bit flags (**zero_f, inf_f, nan_f, tiny_f, huge_f, inexact_f**) that correspond to 6/8 of the status bits. These bits have to be combined after the exception handling in the main module in order to create the '**status**' output. The block diagram of the exception adder is depicted in Figure 9.

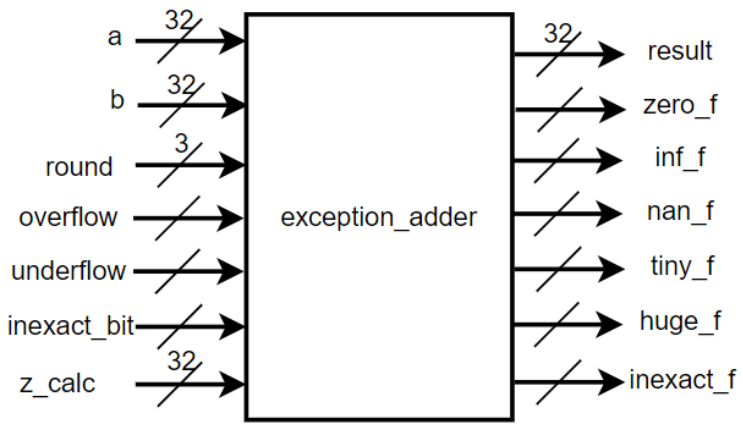

Figure 9: The block diagram of the ***exception _adder*** module

Inside the module you need to create an enumerated typedef called '**interp_t**' (interpreter), two helpful functions that will use this interpreter and a case block.

The enumerator '**interp_t**' will contain all the possible unsigned groups that the floating point number can be part of, as seen in Figure 3, as well as the minNormal and maxNormal cases. These are ZERO, INF, NORM, MIN_NORM, and MAX_NORM. The groups 'NaNs' and 'Denormals' do not need to be added, since they are considered infinities and zeroes respectively.

The first function is called '**num_interp**' (numeric interpretation) and will return an enum type from the enumerator **interp_t** based on the value of a 32-bit input logic signal. Be careful to consider denormals as zeroes, and NaNs as infinities at this point. The MIN_NORM and MAX_NORM do not need to be included in the function.

The second function called '**z_num**' will do the opposite, so it will take an enum value from the enumerator **interp_t** and based on that, will return the unsigned version (31 bits) of the value. For example, if the enumerator has the value 'ZERO' it will return 31 zero bits. The NORM value does not need to be included in this function.

Finally, inside an always_comb block, initialize the values for the status bit outputs of the module into zeroes. Then based on Table 4, define the value of '**z**' accordingly as well as activate the necessary **status bits**. The sign of 'z_calc' as well as the functions you defined above will help you in this case.

$(-1)^{SA}\ 2^{EA}\ 1.MA$	$(-1)^{SB}\ 2^{EB}\ 1.MB$	$(-1)^{SZ}\ 2^{EZ}\ 1.MZ$
+/- ZERO	+/- ZERO	+/- ZERO
+/- ZERO	+/- INF	+/- INF
+/- ZERO	+/- NORM	+/- NORM
+INF	-INF	NAN
+ /- INF	+/- NORM	+/- INF

Table 5: Possible corner case combinations for a and b signals. Combination Norm x Norm is not included in the table

For the **Normal x Normal** combination, you **need to check** in case there is ***overflow*** or ***underflow***. If there is an *overflow*, then based on the 'round' input and the sign of the 'z_calc' signal, round the output either to maxNormal number or to infinity. Likewise, if there is an *underflow*, then based on the same things, round the output either to minNormal or to zero. If there is neither overflow nor underflow, then the 'z' should be equal to 'z_calc', and the 'inexact_f' should be equal to 'inexact'.

4.4 Floating Point Adder – Verification

The verification process will contain a testbench, where the DUT results will be compared to a reference model. Thus, a test-vector generator will be responsible to feed the testbench and the reference model with random and corner gerenated values, and then it will compare their outputs. In addition, two module with assertions are requested to be implemented to check the correctness of flags along the produced outputs. The structure of the testbench is presented in Figure 10.

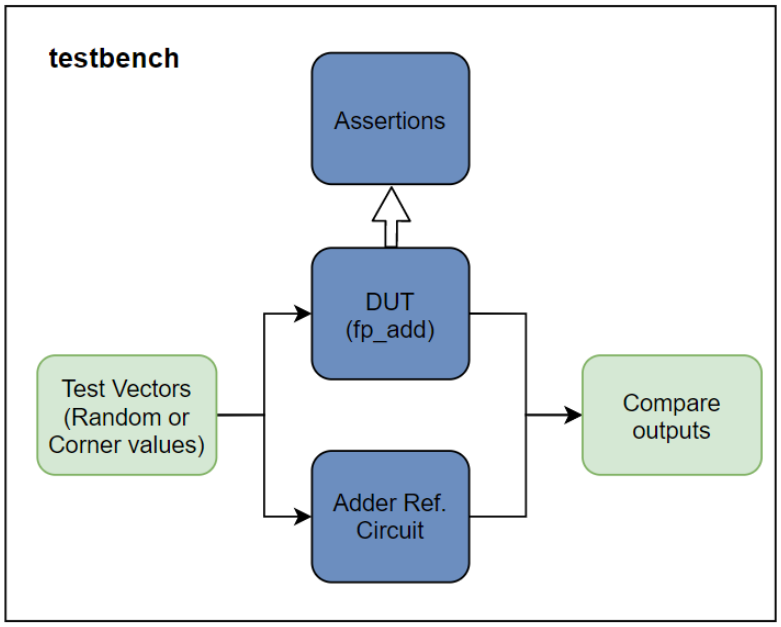

Figure 10: Testbench block diagram structure

4.4.1 Testbench

The second task of the coursework requires you to write a testbench in System Verilog to test the correctness of the modules you have created in the previous exercise <u>for all 5 possible roundings</u>. The testbench will have two parts, in the first part you will be checking the addition result of random values for inputs, while in the second part, you will be checking the result of the corner cases. Choose the **clock period** equal to **10ns**.

For the first part, you may use the '*$urandom()*' function to produce random values for the inputs 'a' and 'b'. You must compare the 'result' of the wrapper with the actual result of the addition and display **ONLY THE ERRORS in case there is a mismatch between the two.**

For the second part, you have to prove the correctness of the addition of all the possible corner cases for the two inputs 'a' and 'b' as defined in Figure 3. It is advised to create a function in this case that matches each corner case with a respective **enum type** since it will be helpful (*i.e.*, match the binary of positive infinity with a type called 'pos_inf'). As corner cases for each of the 'a' and 'b' inputs choose the 10 written below:

- 1 random negative and 1 random positive NaN
- negative and positive infinity
- 1 random negative and 1 random positive normal (parameterized and generated by using the *$urandom()* function)
- 1 random negative and 1 random positive denormal (parameterized and generated by using the *$urandom()* function)
- negative and positive zero

You have to test all the possible combinations of the corner cases between the 'a' and 'b' signals (10x10 = 100 combinations) and check if the result of the wrapper is equal to the result of the *reference model*. It is recommended to use **two separate tasks,** one for the random test generation and the other for the corner test generation, to produce a better.

At the end of the simulation you need to display the total tests that have been executed, the number of SUCCESS random tests over the total number of random tests, and the number of SUCCESS corner tests over the total number of corner tests. Import a screenshot of these statistics in the pdf report.

Warning: Despite the circuit's two-cycle delay**, the testbench must supply each batch of input vectors every cycle**, avoiding stalls while waiting for results. Thus, you will need to align the produced results of the DUT, which have a delay, with the output reference results, which produced from a combi circuit. It is recommended to change the input vector values at the **negative edge of clock**.

4.4.4.1 Reference model

As for the reference model, the Hardfloat open source library will be utilized [7]. From the Hardfloat you need to instantiate the ***addRecFN*** module. According to the reference (here: https://www.jhauser.us/arithmetic/HardFloat-1/doc/HardFloat-Verilog.html), the hardfloat floating point modules are parameterized for the length of exponent and significand bits.

Warning: See the sub-paragraph 5.1 for the format of expWidth and sigWidth, based on 32-bit precision.

The inputs and outputs of Hardfloat modules are recoded formats (see paragraph 5, in the documentation). Thus, you need to use the ***fNToRecFN*** function, to convert the ***a, b*** floating point inputs to the Hardfloat recoded inputs and the ***recFNToFN*** function to convert the Hardfloat output into the corresponding floating point output.

Warning: The recoded Hardfloat values have 33-bit length for single precision !!

The Hardfloat library has been constructed to support fully the IEEE-754 standard, including the handling of denormals and NaNs. To comply with the simpler format of standard, use the template tb_adder.sv file, which contains a combinational block to handle the denormals and NaNs inputs and outputs. Thus, the ***a_hf*** and ***b_hf*** will be the floating point vectors to be fed into the recoded modules and ***results_hf*** will be the 32-bit hardfloat value, after the recoding, which produces the final, IEEE compliance = 0, ***results_ref*** floating point signal.

Warnings:

1) *It is strongly recommended to use the Hardfloat files that have been uploaded to eLearning, to avoid compile issues*
2) *It is strongly recommended to use the Questa environment, because the EDA playground has a bottleneck in design size and possibly it will be an issue with the waveforms that you can generate*
3) *If you are working in Questa, you can import ALL the files in your project as typical Verilog files.*
4) *If you are working in EDA playground, again upload the ALL the files. It is recommended to use a filelist which will contain all the file names and then to use for example the flag –f filelist.f in the compile options, if you are using the Cadence compiler (similarly you can find the appropriate flag in other compilers).*

4.4.2 System Verilog Assertions

Based on Table 3, you may have realized that some of the status bits cannot assert at the same time. The first part of this task is to write all possible ***Immediate Assertions***, testing each time that **two** specific status bits do not assert to 1 together. You need to identify which combinations of two status bits **should never** assert at the same time, based on the descriptions given above.

For the second part, you need to write four ***Concurrent Assertions***. These assertions will be responsible for checking the following things (at each positive clock edge):

- If the '**zero**' status bit asserts to 1 then at the same cycle all the bits of the exponent of 'z' must be equal to 0.
- If the '**inf**' status bit asserts to 1 then at the same cycle all the bits of the exponent of 'z' must be equal to 1.
- If the '**nan**' status bit asserts to 1 then 3 cycles before all the bits of the exponent of 'a' and 'b' must be equal to 1 and the 'a' and 'b' signals have opposite signs.
- If the '**huge**' status bit asserts to 1 then at the same cycle all the bits of the exponent of 'z' must be equal to 1, or all the bits of the exponent of 'z' except the LSB must be equal to 1, the LSB must be 0 and all the bits of the mantissa of 'z' to be equal to 1 (maxNormal case).

Implement the two parts in two different modules called '**test_status_bits**' and '**test_status_z_combinations**' respectively, and then bind the two modules with the wrapper module.

5 Deliverables for Submission

For the floating point adder, discussed in Section 3 and 4, you SHOULD:

- Produce the SystemVerilog code for the modules ***fp_adder***, ***exponent_calc***, ***mantissa_calc***, ***norm_adder***, ***round_adder***, ***exception_adder*** and ***lzc*** (**7** .sv files + 1 .sv file for the round_pkg)
- Produce a **testbench** that demonstrates the correct operation of the circuit as well as the correct operation of the corner cases (**1** .sv file) (BASED ON ref. tb_adder.sv)
- Using SVA, provide the necessary properties that produce proper messages on the simulator Transcript window (std out) to report on the PASS/FAIL of the additional checks required. (**1** .sv file)

You should prepare a report (in **pdf** format) that will describe in more detail your thought process, what you have designed and how you have verified the DUT, as well as contain screenshots from the respective simulations. If you add any more modules that are not mentioned in the coursework, you should describe these modules and their operation in your pdf report. Submit these folders along with the pdf report in a zip file named *<lastname_AEM>.zip* on elearning.

Warning: Please do not include greek characters in the names of your folders since elearning changes them into something incoherent.

The coursework takes 3 marks out of the total mark (i.e., 30%) and is **compulsory**. If the coursework is not submitted on time, no marks are given but you can take the exam. In this case, the maximum mark that can be achieved is 7. The mark for the coursework is maintained for the next two exam periods ONLY (Sept. 2026, Feb. 2027).

The submission deadline will be until 21/6/2026.

Good Luck!!

6 References

[1]. ModelSim [Online]. Available: https://eda.sw.siemens.com/en-US/ic/modelsim/

[2]. Questa*-Intel® FPGA Edition [Online]. Available: https://www.intel.com/content/www/us/en/software-kit/684216/intel-quartus-prime-lite-edition-design-software-version-21-1-for-windows.html Libero [Online]. Available: https://www.microchip.com/en-us/products/fpgas-and-plds/fpga-and-soc-design-tools/fpga/libero-software-later-versions

[3]. https://www.intel.com/content/www/us/en/docs/programmable/683472/22-1/minimum-hardware-requirements.html

[4]. Intel® FPGA Self Service Licensing Center (SSLC) [Online]. Available: https://tinyurl.com/yc8f8es4

[5]. IEEE Std 754-2019, "IEEE Standard for Floating-Point Arithmetic," IEEE, New York, NY, USA, 2019.

[6]. Hardfloat [Online]. Available: https://www.jhauser.us/arithmetic/HardFloat.html

7 Appendix

The following figures help you understand the simulation tool and should be read with subsection 3.1.2.

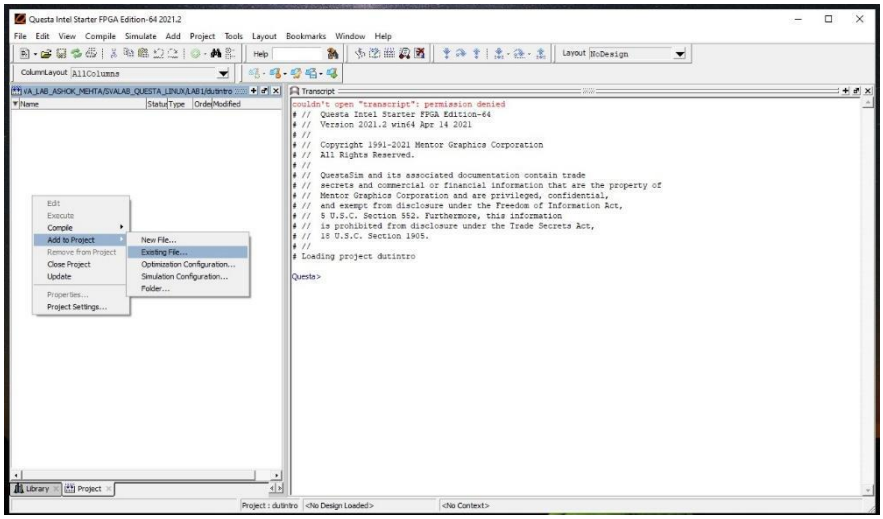

Fig. 6. Started a new project and ready to add files (right click in the project area for these menus to appear).

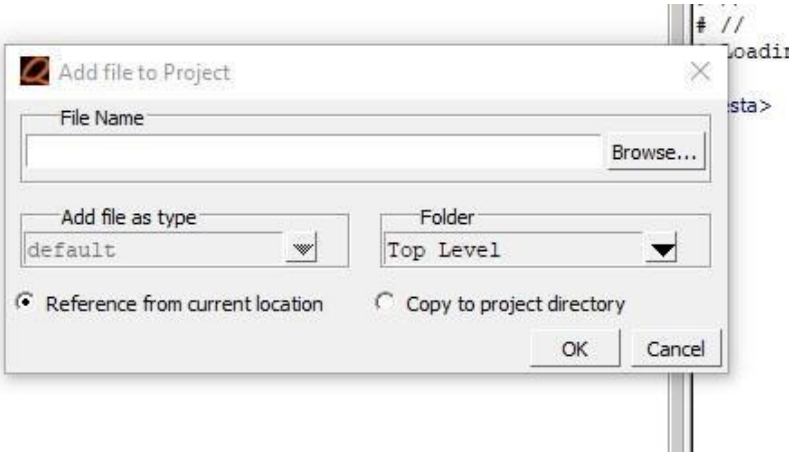

Fig. 7. Browse and pick the files you need (copy or not the files to the project directory).

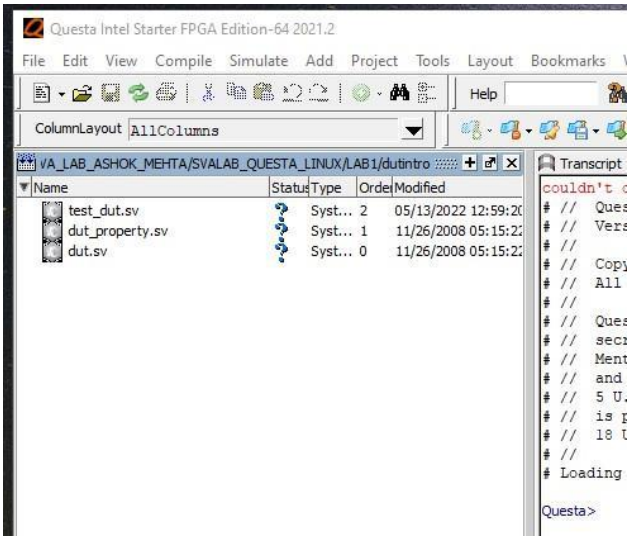

Fig. 8. Files were added to the project and are now ready to be compiled.

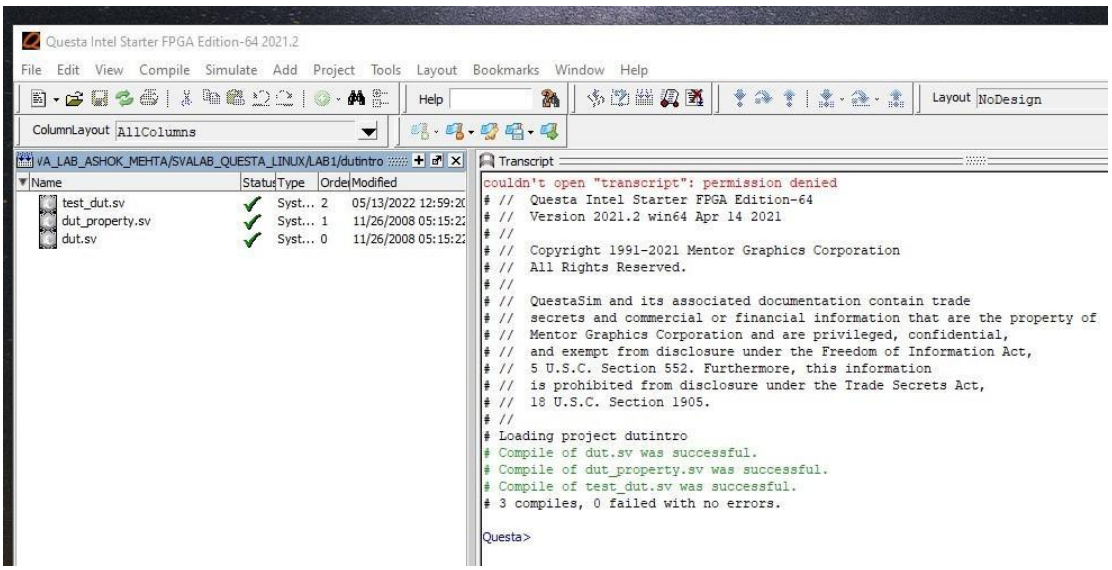

Fig. 9. All files have now been complied (question marks turns to ticks).

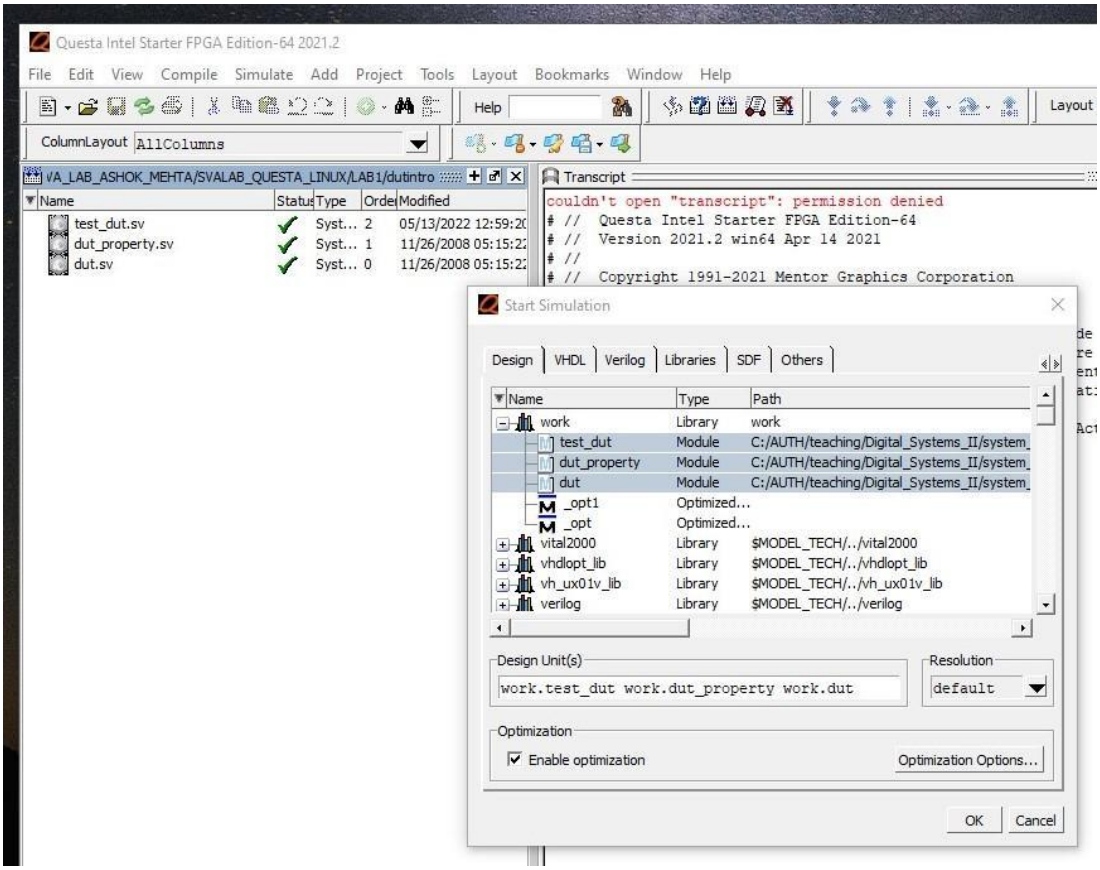

Fig. 10. Click on Simulate -> Start Simulation and select the compiled files to be simulated. Click on optimization options (see next Figure).

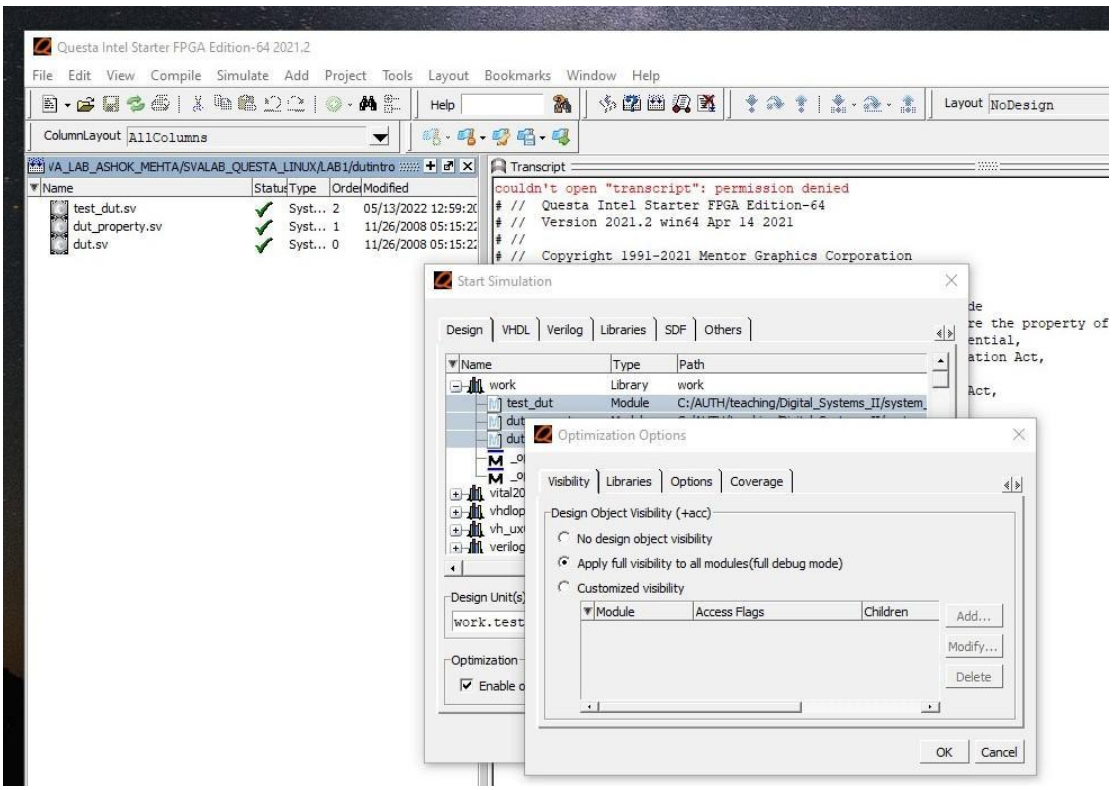

Fig. 11. Full visibility allows you to display all internals signals in the waveform window.

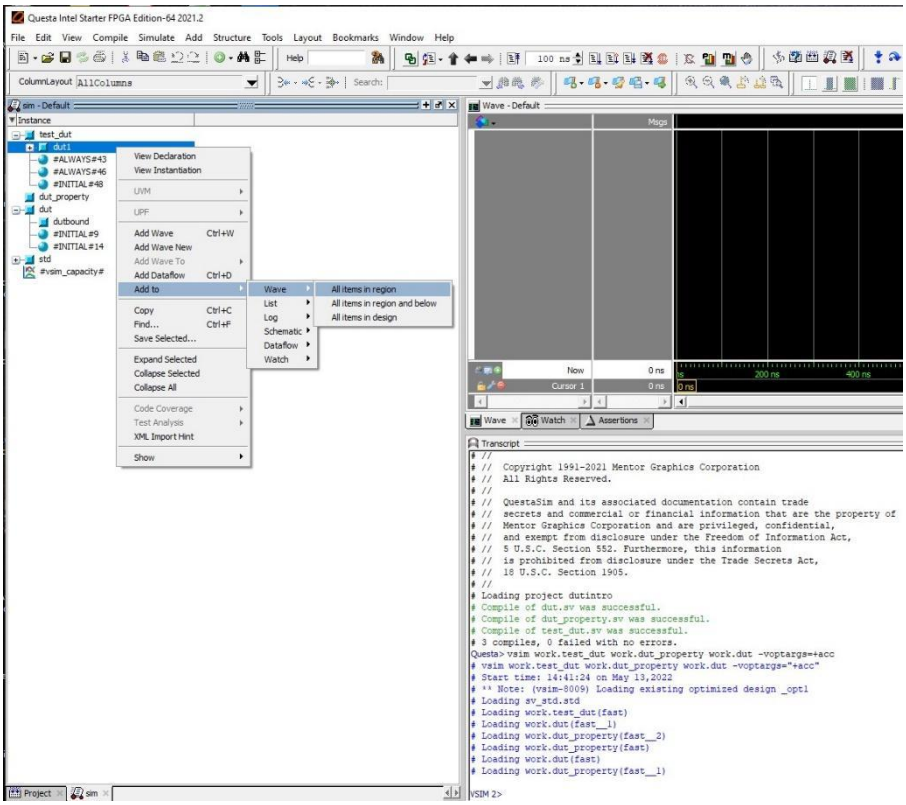

Fig. 12. Once the files are loaded for simulation, you can select which signals to be displayed (right click).

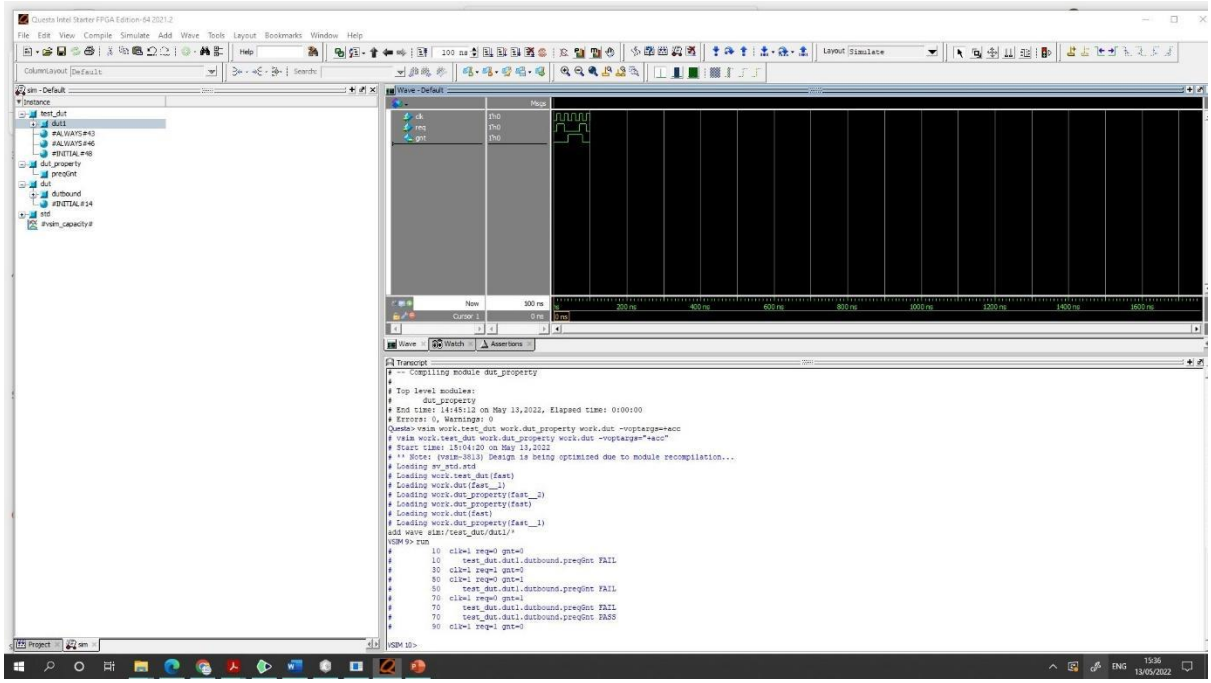

Fig. 13. Simulated DUT. See also the simulator transcript window at the bottom and the related reports.

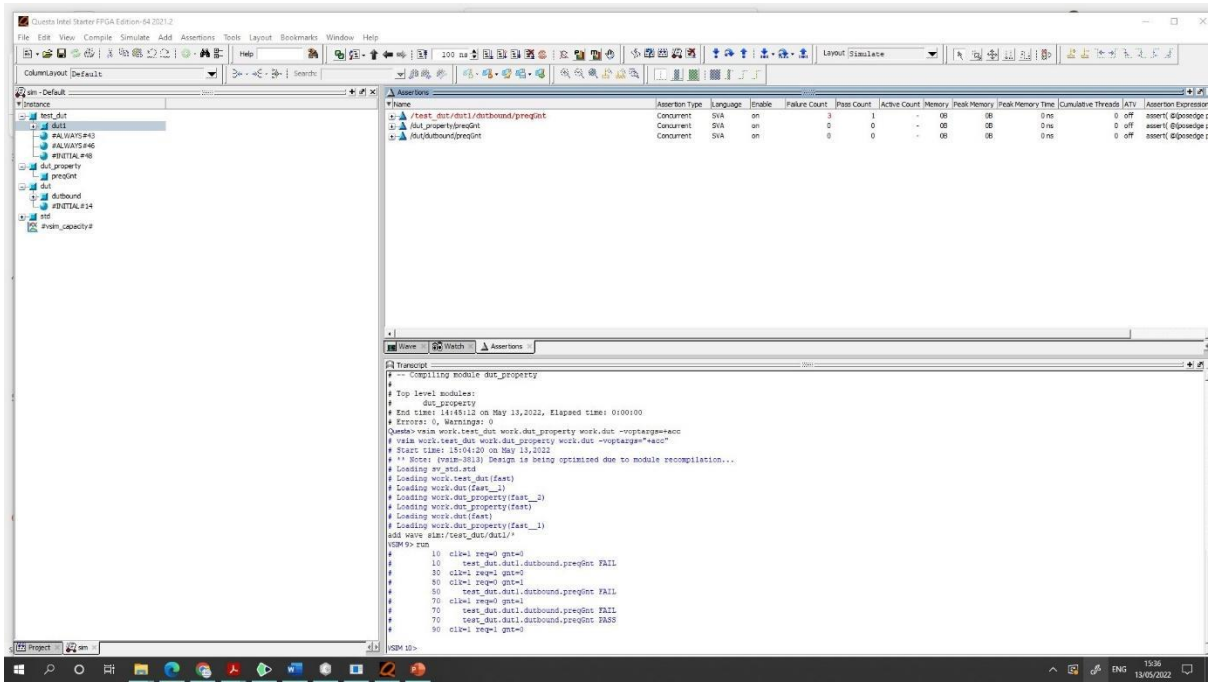

Fig. 14. Simulated DUT. The Assertions tab displays all related info about the properties checked in file dut_property.sv